

Multi-Line Insurance Policy and Claims Management System

Salesforce Developer Capstone

Student & Project Details

Project Title	Multi-Line Insurance Policy and Claims Management System
Name	S.SARAN KUMAR
Reg No	820323205094
Institution	A.V.C.COLLEGE.OF.ENGINEERING
NM ID	B963FFF243624BB8A04473FAA5C0FFB5

Abstract

Insurance carriers that sell vehicle, property and life cover often run each line on separate, partly manual processes. Quotes take long to produce, claims are reviewed by hand, and nobody sees a customer's policies and claims together. This project delivers a single Salesforce solution for a multi-line carrier. It combines a flexible custom data model (Policy and Claim objects with Record Types), guided Screen Flows for quoting, Apex-based premium rating, Record-Triggered Flows for claim routing and approval, and a Lightning Web Component dashboard for claims adjusters. Validation rules, sharing rules and permission sets keep data accurate and access appropriate for agents, adjusters and managers. The work was delivered in four milestones, described in this report.

1. Introduction

1.1 Problem Statement

The carrier manages vehicle, property and life insurance on a legacy system that makes quoting, issuance and claims handling manual, slow and inconsistent. Without a unified platform, cross-selling is difficult, and manual claim review lengthens resolution times, raises operating costs and weakens the customer experience. The business needs one Salesforce solution that can:

- **Standardise and accelerate** quoting and policy issuance for every insurance line.
- **Automate claim initiation and routing** by policy type and claim severity.
- Provide a **360-degree view** of Customer, Policy and Claim data in one place.

1.2 Objectives

- Design a flexible data model that stores diverse product data, such as VIN for vehicles and square footage for property.
- Implement Screen Flows that guide agents through multi-step quoting for each policy type.
- Develop a dynamic Apex/LWC Claims Dashboard so adjusters can manage their workload efficiently.
- Automate claim approval and escalation by claim amount using Record-Triggered Flows.
- Enforce data validation across all insurance products for compliance and accuracy.

1.3 Challenges and How They Were Addressed

Challenge	Description	Approach taken
Variable data management	Vehicle, Property and Life policies need different fields within one object structure.	Single Policy object with Record Types and Field Sets per line.
Approval complexity	Approval must route by policy type, claim amount and adjuster level.	Amount-based entry criteria with a two-step sequential approval process.
API integration simulation	Quote verification against an external vehicle or property database.	Apex rating logic built as an invocable action; callout mocking planned (Section 7).
User adoption	Quoting must need few clicks to keep agents efficient.	Short, focused multi-screen Screen Flow with guided inputs.

2. Solution Overview and Architecture

The solution uses declarative tools wherever possible and code only where logic or performance requires it. Screen Flows handle interactive quoting, a custom data model supports policy diversity, Apex provides rating logic and data access, Record-Triggered Flows automate routing and submission, and LWC delivers a focused adjuster dashboard.

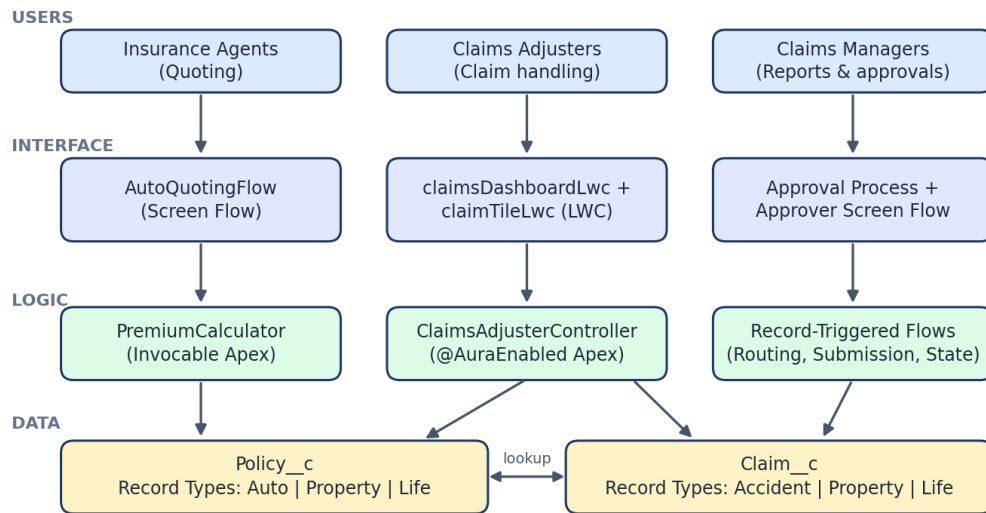


Figure 1: Layered architecture of the Multi-Line Insurance Policy and Claims Management System

Component	Type	Purpose
Policy__c / Claim__c	Custom objects	Core records with Auto Number names (P-0000, C-0000) and Record Types.
AutoQuotingFlow	Screen Flow	Guides agents through Auto policy quoting and creates a draft policy.
PremiumCalculator	Apex (Invocable)	Rates a policy by state and model year; updates the draft premium.
Claim routing flow	Record-Triggered Flow	Assigns each new claim to the Auto, Property or Life queue.
claimsDashboardLwc, claimTileLwc	Lightning Web Components	Adjuster dashboard with client-side filtering and reusable claim tiles.
High Value Claim Approval	Approval Process + Flows	Two-step approval for claims above \$50,000.

3. Data Model

Both main objects use Auto Number record names, have Reports, Activities and Field History Tracking enabled, are deployed, and allow search. Record Types separate the three insurance lines, while Field Sets group the fields each line needs.

3.1 Policy Object (Policy__c)

Field	Data type	Used by
Customer	Lookup (Contact)	All policy types
Policy Start Date	Date	All policy types
Premium	Currency (16, 2)	All; set by PremiumCalculator
VIN	Text (20)	Auto (validated to 17 characters)
Model Year	Text (5)	Auto
Square Footage, Year Built	Number (18, 0), Text (5)	Property
Beneficiary Name, Policy Term Months	Text (50), Number (18, 0)	Life

Record Types **Auto**, **Property** and **Life** are active and available to all profiles. The Field Sets are Vehicle (VIN, Model Year), Property (Square Footage, Year Built) and Life (Beneficiary Name, Policy Term Months).

3.2 Claim Object (Claim__c)

Field	Data type	Purpose
Policy	Lookup (Policy)	Links the claim to its policy and customer.
Claim Amount	Currency (16, 2)	Drives approval and escalation.
Date of Loss	Date/Time	Used to calculate days open.
Approval Status	Picklist: New, Submitted for Approval, Approved, Rejected	Tracks the claim through approval.
Adjuster	Lookup (User)	Identifies the responsible adjuster.
Description	Long Text Area	Details of the loss.

The Claim object has Record Types such as **Accident**, with matching Property and Life types following the same configuration.

4. Implementation by Milestone

4.1 Milestone 1: Core Data Model and Policy Configuration

This milestone created the Policy and Claim objects, their Record Types, Field Sets and fields (Section 3). It then added the quoting flow and a validation rule.

- **AutoQuotingFlow (Screen Flow).** A Get Records element retrieves the Auto Record Type ID. Screen 1 captures the policy holder (Contact lookup), start date and policy state. Screen 2 captures the VIN and model year. A Create Records element then saves a draft Policy and stores its ID in the variable varPolicyId.
- **Validation rule VIN_Must_Be_17_Characters.** The formula `AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)` blocks any Auto policy whose VIN is not exactly 17 characters, and shows the error on the VIN field.

4.2 Milestone 2: Complex Policy Issuance and Claim Routing

Premium calculation. The invocable Apex class PremiumCalculator accepts a list of policy IDs, queries VIN, Model Year and Policy State, and returns a premium for each. The flow calls it after the draft policy is created, stores the result in varCalculatedPremium, and an Update Records element writes it to the Premium field.

Rating rule	Effect on premium
Base premium	1,000.00
Policy State = CA	Multiply by 1.15
Policy State = TX	Multiply by 1.05
Model Year earlier than 2018	Add 100

Claim routing. A Record-Triggered Flow (after save, on create) on Claim retrieves the related policy, then a Decision element, Route by Policy Type, sends the claim down the Auto, Property or Life path. Each path updates the claim Owner to the matching queue, so new claims reach the correct team without manual triage.

4.3 Milestone 3: Claims Adjuster LWC Dashboard

- **ClaimsAdjusterController (Apex).** The cacheable @AuraEnabled method getAssignedClaims queries all claims owned by the logged-in user, with related Policy and Customer data. It returns a ClaimWrapper list holding claim ID and number, amount, status, policy type, policy holder name and days open.
- **claimsDashboardLwc.** Loads the data through the @wire service and offers a combobox filter (All, Auto, Property, Life). Filtering runs client-side on the wired data, so changing the filter needs no server call. It shows the total number of claims assigned.
- **claimTileLwc.** A reusable child component that receives one claim through an @api property and shows the claim number, policy holder, amount, status and days open. It chooses an icon by policy type.

4.4 Milestone 4: Advanced Claim Processing, Security and Code Setup

- **High Value Claim Approval process.** Applies to claims with Claim Amount above 50,000. Step 1 goes to the Senior Adjuster and Step 2 to the Department Manager. Field updates set Approval Status to Submitted for Approval, Approved or Rejected at the matching stage.
- **Submission Automation Flow.** A Record-Triggered Flow on create or update checks for Claim Amount above 50,000 and Approval Status = New, then submits the record to the approval process.
- **Claim Approver Screen Flow.** Shows the claim ID, amount and policy type, collects an Approve or Reject decision plus comments, and is exposed on the Claim page as the quick action Approve/Reject Claim.
- **Territory visibility.** The flow Claim Policy Holder State Update copies the policy state onto the claim. Criteria-based sharing rules then give Read/Write access for claims in CA and TX.
- **Test class.** ClaimsAdjusterControllerTest builds a test user, Contact, Auto Policy and Claim, then runs getAssignedClaims as that user and asserts the returned wrapper values.

5. Security and Access Control

Three Permission Sets separate duties between agents (quoting only), adjusters (claim handling) and managers (reporting and approvals). Territory-based sharing rules add record-level control on top of these object permissions.

Permission Set	Policy	Contact	Claim	System / Apex access
Insurance Agent Access	Create, Read	Read, Edit	Read	Flow User enabled; PremiumCalculator
Claims Adjuster Access	Read	n/a	Read, Edit	ClaimsAdjusterController
Claims Manager Access	Read	Read, Edit	Read, Edit, Delete	Run Reports; Manage Approvals

6. Testing and Validation

Area	Check performed	Expected result
Validation rule	Save an Auto policy with a VIN not 17 characters long.	Save blocked; error shown on the VIN field.
Quoting flow	Complete both screens for an Auto quote.	Draft policy created; premium populated by Apex.
Claim routing	Create a claim on an Auto, Property and Life policy.	Owner set to the matching queue.
Approval	Create a claim with amount above 50,000.	Submitted automatically; two-step approval; status updated.
Dashboard	Open as an adjuster and change the filter.	Only own claims shown; list narrows by policy type.
Apex test	Run ClaimsAdjusterControllerTest.	Assertions on claim ID, number, status, amount and holder name pass.

7. Observations and Recommended Improvements

Review of the implementation against its stated objectives identified refinements that would harden the solution before production use.

Area	Observation	Recommendation
Days open	daysOpen is computed as <code>Date.today().daysBetween(dateOfLoss)</code> , which gives a negative number for past dates.	Use <code>dateOfLoss.date().daysBetween(Date.today())</code> .
Dashboard query	The wrapper reads <code>RecordType.Name</code> and <code>Contact Name</code> , but the SOQL selects <code>RecordType.DeveloperName</code> , <code>FirstName</code> and <code>LastName</code> .	Select every field the wrapper reads, or build names from <code>FirstName</code> and <code>LastName</code> .
Routing logic	The routing flow stores a Record Type ID in <code>varPolicyRTName</code> but compares it with the names <code>Auto</code> and <code>Property</code> .	Resolve the Record Type name first (Get Records on <code>RecordType</code> or a formula field).
Queue IDs	Queue IDs are hardcoded in the routing flow.	Look up queues by <code>DeveloperName</code> , so flows move between sandboxes and production.
Field coverage	Policy State and Policy Account Holder State are used by the flows, Apex and sharing rules but are not in the field tables.	Document and create both fields before deployment.
Approver decision	The approver flow uses the Submit for Approval action to record Approve and Reject.	Record decisions through an Apex invocable action using <code>ProcessWorkitemRequest</code> .
Integration and tests	The external verification callout is not implemented, and only one test method exists.	Add an <code>HttpCalloutMock</code> -based callout and tests for filtering, empty results and other policy types.

8. Expected Benefits

- **Faster quoting and issuance:** a single guided flow replaces manual, inconsistent steps and calculates the premium automatically.
- **Shorter claim resolution:** automatic routing and amount-based approval remove manual triage and escalation.
- **Unified customer view:** policies and claims connect through lookups, which supports cross-selling across lines.
- **Better data quality and security:** validation rules, sharing rules and permission sets enforce accuracy and least-privilege access.

9. Future Enhancements

- **External verification:** add an Apex callout to a motor-vehicle or property database for quote verification, with an HttpCalloutMock test class.
- **Quoting for all lines:** extend the Screen Flow to Property and Life policies using the existing Record Types and Field Sets.
- **Richer rating:** move rating factors into Custom Metadata so underwriters can change them without code.
- **Escalation tiers:** add adjuster-level and policy-history rules to the approval routing.
- **Reporting:** build Salesforce dashboards for claim cycle time, open claims by queue and premium by line.

10. Conclusion

The project delivers a unified Salesforce platform for vehicle, property and life insurance. A flexible Policy and Claim data model, guided quoting, Apex premium rating, automated routing and approvals, and an LWC adjuster dashboard together meet the stated objectives of standardised quoting, faster claims handling and a single view of the customer. The improvements in Section 7 and the enhancements in Section 9 are the natural next steps toward a production-ready release.